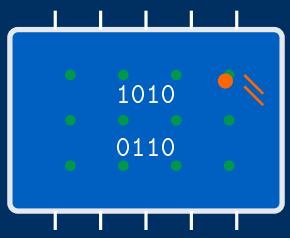
Université Gustave Eiffel

Master 2 MCC

Microsystèmes et Capteurs Communicants



PROJET FPGA

Jeu Électronique de Type

« *Simon* »

Implémentation Matérielle sur FPGA

Programmation

- VHDL-93
- Logique Séquentielle
- Machine à États

Électronique

- FPGA Zynq-7000
- LEDs & Switches
- Interface I/O

Réalisé par :

Mohamed SELMANI

Mohamed ZAGHOUANI

Lynda RAHMANI

Encadré par :

Mme DAM Thi Hong

Mme Maha BEN RHOUMA

Environnement :

- FPGA Zynq-7000
- Xilinx Vivado
- VHDL-93 / XDC

Année Universitaire 2025 - 2026

Contents

1 Introduction **5**
1.1 Contexte du Projet . . . 5
1.2 Objectifs Pédagogiques . . . 5
1.3 Le Jeu Simon . . . 6
1.3.1 Principe Original . . . 6
1.3.2 Notre Implémentation . . . 6

2 Architecture Matérielle **7**
2.1 Composants de la Carte FPGA . . . 7
2.1.1 Entrées . . . 8
2.1.2 Sorties . . . 8
2.2 Fichier de Contraintes . . . 8
2.2.1 Rôles du Fichier de Contraintes . . . 8
2.2.2 Structure des Contraintes . . . 9
2.2.3 Configuration de l'Horloge . . . 9
2.2.4 Standard LVCMOS33 . . . 9
2.2.5 Conséquences d'un Fichier XDC Incorrect . . . 10

3 Analyse Détaillée du Code VHDL **11**
3.1 Structure Générale . . . 11
3.2 Déclarations et Bibliothèques . . . 11
3.3 Entity - Interface Externe . . . 11
3.3.1 Paramètres Génériques . . . 12
3.4 Constantes Calculées . . . 12
3.5 Signaux Internes . . . 13
3.5.1 Signaux de Synchronisation . . . 13
3.5.2 Signaux d'Antirebond . . . 13
3.5.3 Signaux de Détection de Front . . . 14
3.5.4 Machine à États . . . 14
3.5.5 Signaux pour le LFSR et la Séquence . . . 14
3.6 Processus 1 : Anti-Métastabilité . . . 14
3.7 Processus 2 : Antirebond (Debounce) . . . 15
3.8 Processus 3 : Détection de Front Montant . . . 16
3.9 Processus 4 : LFSR et Machine à États . . . 17
3.9.1 LFSR (Linear Feedback Shift Register) . . . 17
3.9.2 Gestion des États . . . 18

3.10 Affectations de Sortie 20

4 Schematic - Vue Logique du Circuit 21
4.1 Présentation du Schematic 21
4.2 Rôle et Importance du Schematic 21
4.2.1 Vérification de la Synthèse 21
4.2.2 Éléments Visibles dans le Schematic 22
4.2.3 Analyse du Schematic 22

5 Concepts Avancés de Conception Numérique 23
5.1 Synchronisation et Métastabilité 23
5.1.1 Le Problème de la Métastabilité 23
5.1.2 Solution : Double Synchroniseur 23
5.2 Antirebond (Debouncing) 24
5.2.1 Phénomène de Rebond 24
5.2.2 Algorithme d'Antirebond 24
5.3 Générateurs de Nombres Pseudo-Aléatoires 24
5.3.1 LFSR : Théorie 24
5.3.2 Implémentation Matérielle 25
5.4 Machines à États Finis (FSM) 25
5.4.1 Diagramme d'États Complet 25
5.4.2 Types de FSM 25

6 Validation et Tests 26
6.1 Simulation Fonctionnelle 26
6.1.1 Banc de Test (Testbench) 26
6.2 Tests sur Carte FPGA 27
6.2.1 Procédure de Test 27
6.2.2 Scénarios de Test 28
6.3 Analyse des Ressources 28

7 Améliorations et Extensions Possibles 29
7.1 Fonctionnalités Supplémentaires 29
7.1.1 Niveaux de Difficulté 29
7.1.2 Séquence Progressive 29
7.1.3 Compteur de Score 29
7.1.4 Mode Multi-joueur 29
7.2 Optimisations Techniques 30
7.2.1 LFSR Plus Complexe 30
7.2.2 Communication UART 30
7.2.3 Sons et Effets 30
7.3 Interface Graphique 30
7.3.1 Affichage VGA 30

8 Conclusion **31**
8.1 Synthèse du Projet 31
8.2 Compétences Acquises 31
8.2.1 Compétences Techniques 31
8.2.2 Compétences Transversales 32
8.3 Applications Pratiques 32
8.4 Perspectives 32
8.5 Réflexion Personnelle 33

A Code VHDL Complet **34**
A.1 Fichier Principal : jeu_led.vhd 34

B Fichier de Contraintes **39**
B.1 Fichier XDC Complet 39

C Glossaire **41**

D Bibliographie **42**

List of Figures

1.1 Diagramme de flux du jeu Simon 6

2.1 Architecture système complète du projet FPGA 7

3.1 Flux de traitement des signaux : synchronisation, débounce et détection de front 13

3.2 Détection de front montant - Condition et table de vérité 17

4.1 Schematic généré par Xilinx Vivado - Vue logique RTL du circuit 21

5.1 Phénomène de rebond mécanique 24

5.2 Diagramme de la machine à états finis du jeu Simon 25

8.1 L'équipe du projet pendant la phase de développement et de tests sur FPGA 33

Chapter 1

Introduction

1.1 Contexte du Projet

Ce projet s'inscrit dans le cadre de l'apprentissage de la conception de systèmes numériques sur FPGA (Field-Programmable Gate Array). L'objectif est de concevoir et d'implémenter un jeu électronique inspiré du célèbre jeu **Simon**, en utilisant le langage de description matériel **VHDL** (VHSIC Hardware Description Language).

Qu'est-ce qu'un FPGA ?

Un FPGA est un circuit intégré reconfigurable qui permet d'implémenter des circuits numériques personnalisés. Contrairement aux processeurs classiques qui exécutent des instructions séquentiellement, un FPGA permet de créer des circuits matériels parallèles hautement optimisés.

1.2 Objectifs Pédagogiques

Ce projet permet d'aborder plusieurs concepts fondamentaux de la conception numérique :

- ✓ **Logique Combinatoire** : Opérations logiques, comparaison de signaux
- ✓ **Logique Séquentielle** : Registres, machines à états finis (FSM)
- ✓ **Gestion Temporelle** : Compteurs, divisions d'horloge, timing
- ✓ **Génération Pseudo-aléatoire** : LFSR (Linear Feedback Shift Register)
- ✓ **Traitement des Entrées** : Antirebond, anti-métastabilité
- ✓ **Interface Utilisateur** : LEDs, boutons, interrupteurs

1.3 Le Jeu Simon

1.3.1 Principe Original

Le jeu Simon, inventé dans les années 1970, est un jeu électronique de mémoire où le joueur doit reproduire une séquence de couleurs et de sons qui s'allonge progressivement à chaque manche réussie.

1.3.2 Notre Implémentation

Dans notre version simplifiée sur FPGA, le système fonctionne selon le principe suivant :

1. Le système génère une séquence aléatoire de 6 bits
2. Cette séquence est affichée sur les LEDs LD0 à LD5 pendant 2 secondes
3. Le joueur doit mémoriser la combinaison
4. Le joueur reproduit la séquence avec les interrupteurs SW0 à SW5
5. Le joueur valide sa réponse avec le bouton VALID
6. En cas de succès, LED7 clignote 3 fois et une nouvelle manche commence
7. En cas d'échec, le joueur peut réessayer

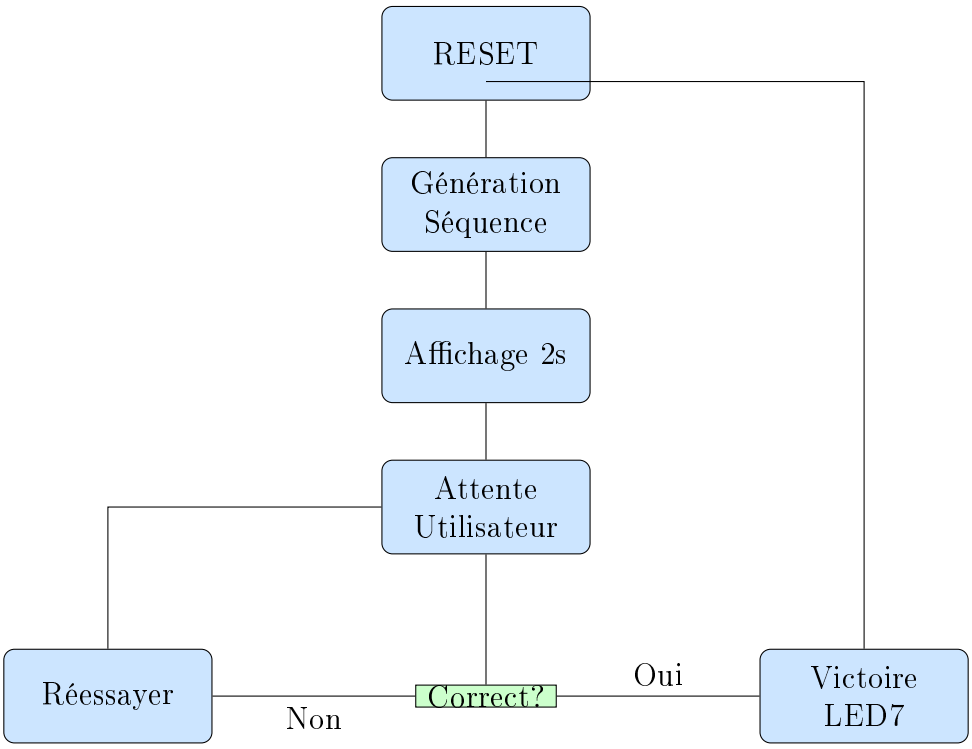


Figure 1.1: Diagramme de flux du jeu Simon

Chapter 2

Architecture Matérielle

2.1 Composants de la Carte FPGA

Notre jeu utilise les composants suivants de la carte de développement FPGA :

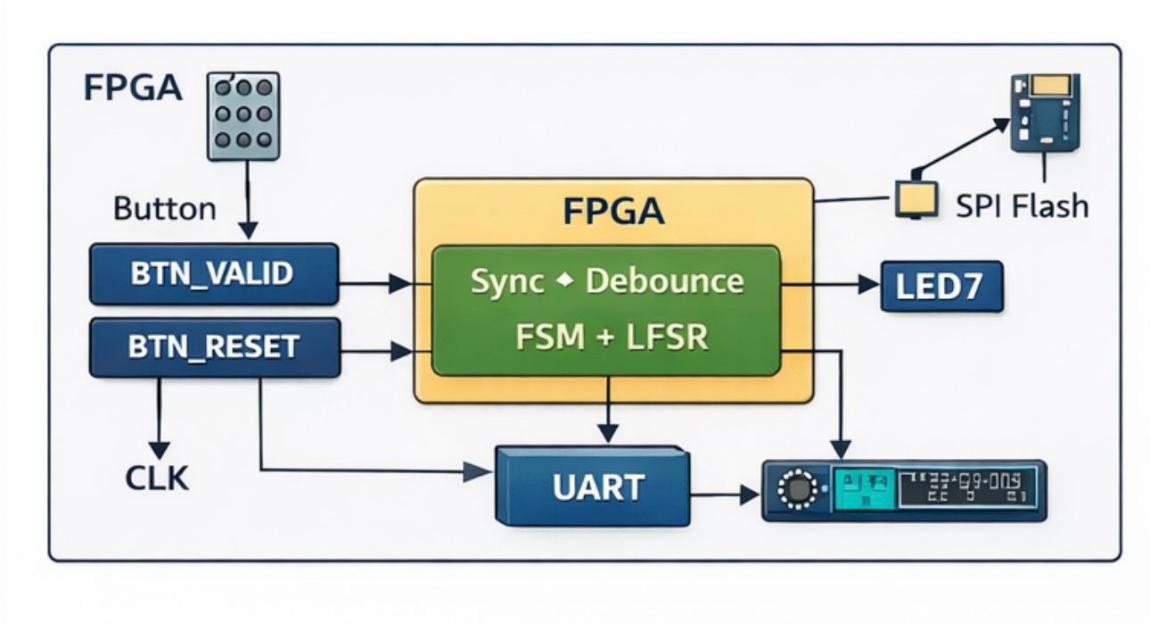


Figure 2.1: Architecture système complète du projet FPGA

2.1.1 Entrées

primaryblue!20 **Composant**	**Broche**	**Description**
CLK	Y9	Horloge système (100 MHz)
BTN_RESET	P16	Bouton de réinitialisation du jeu
BTN_VALID	N15	Bouton de validation de la réponse
SW0	H15	Interrupteur bit 0
SW1	G22	Interrupteur bit 1
SW2	H22	Interrupteur bit 2
SW3	F21	Interrupteur bit 3
SW4	H19	Interrupteur bit 4
SW5	H18	Interrupteur bit 5

Table 2.1: Entrées du système

2.1.2 Sorties

primaryblue!20 **Composant**	**Broche**	**Description**
LED0	T22	LED affichant bit 0 de la séquence
LED1	T21	LED affichant bit 1 de la séquence
LED2	U22	LED affichant bit 2 de la séquence
LED3	U21	LED affichant bit 3 de la séquence
LED4	V22	LED affichant bit 4 de la séquence
LED5	W22	LED affichant bit 5 de la séquence
LED7	U14	LED d'indication de victoire

Table 2.2: Sorties du système

2.2 Fichier de Contraintes

Le fichier de contraintes (XDC - Xilinx Design Constraints) est un élément **absolument essentiel** pour relier les signaux logiques du code VHDL aux broches physiques du FPGA.

Importance Critique du Fichier XDC

Sans fichier de contraintes, le FPGA ne peut pas fonctionner !
Le code VHDL définit la logique du circuit, mais ne spécifie pas où connecter physiquement les signaux. Le fichier XDC associe chaque signal logique (CLK, LED[0], BTN_VALID, etc.) à une broche physique précise du circuit FPGA (Y9, T22, N15, etc.).

2.2.1 Rôles du Fichier de Contraintes

Le fichier XDC remplit plusieurs fonctions critiques :

1. **Association broche-signal** : Définit quelle broche physique du FPGA est connectée à quel signal logique
2. **Standard électrique** : Spécifie le niveau de tension et le type d'interface (LVCMOS33, LVDS, etc.)
3. **Contraintes temporelles** : Peut définir des contraintes de timing pour l'horloge et les chemins critiques
4. **Placement** : Peut forcer certains éléments logiques à des emplacements spécifiques du FPGA

2.2.2 Structure des Contraintes

Chaque contrainte suit la syntaxe TCL (Tool Command Language) de Xilinx :

```
# Associer une broche physique
set_property PACKAGE_PIN <broche> [get_ports <signal>]

# Definir le standard electrique
set_property IOSTANDARD <standard> [get_ports <signal>]
```

Listing 2.1: Structure type d'une contrainte

2.2.3 Configuration de l'Horloge

L'horloge système fonctionne à 100 MHz, ce qui signifie qu'un cycle d'horloge dure 10 nanosecondes.

```
set_property IOSTANDARD LVCMOS33 [get_ports CLK]
set_property PACKAGE_PIN Y9 [get_ports CLK]
```

Listing 2.2: Configuration de l'horloge dans le fichier XDC

2.2.4 Standard LVCMOS33

Toutes les entrées/sorties utilisent le standard **LVCMOS33** (Low Voltage CMOS 3.3V) :

- Tension d'alimentation : 3.3V
- Niveau logique '1' : > 2.0V
- Niveau logique '0' : < 0.8V
- Compatible avec la plupart des périphériques modernes

2.2.5 Conséquences d'un Fichier XDC Incorrect

Des erreurs dans le fichier de contraintes peuvent causer :

- **Échec de l'implémentation** : Si une broche n'existe pas ou est déjà utilisée
- **Comportement erratique** : Si le standard électrique est incompatible
- **Non-fonctionnement** : Si les signaux sont mal connectés
- **Dommages matériels** : Si les niveaux de tension sont incorrects

Vérification du Fichier XDC

Il est crucial de vérifier que :

- Chaque signal d'entrée/sortie a une contrainte PACKAGE_PIN
- Chaque signal a un IOSTANDARD défini
- Les broches correspondent au schéma de la carte (datasheet)
- Aucune broche n'est utilisée deux fois

Chapter 3

Analyse Détaillée du Code VHDL

3.1 Structure Générale

Le code VHDL s'organise en deux parties principales :

1. **Entity** : Interface externe (ports d'entrée/sortie)
2. **Architecture** : Implémentation interne (logique)

3.2 Déclarations et Bibliothèques

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
```

Listing 3.1: Bibliothèques et déclarations

Explication des bibliothèques

- **IEEE.STD_LOGIC_1164** : Définit les types std_logic et std_logic_vector
- **IEEE.NUMERIC_STD** : Fournit les opérations arithmétiques et les conversions de types

3.3 Entity - Interface Externe

```
entity jeu_led is
generic (
    CLK_HZ : integer := 100_000_000;  -- Frequence horloge
    DEBOUNCE_MS : integer := 10;      -- Duree antirebond
    SHOW_TIME_SEC : integer := 2;     -- Temps affichage
    BLINK_PERIOD_MS : integer := 250; -- Periode clignotement
```

```
    BLINK_TOGGLES : integer := 6        -- 6 bascules = 3 clignote
        .
);
port (
    CLK : in std_logic;
    SW : in std_logic_vector(5 downto 0);
    BTN_VALID : in std_logic;
    BTN_RESET : in std_logic;
    LED : out std_logic_vector(5 downto 0);
    LED7 : out std_logic
);
end entity;
```

Listing 3.2: Déclaration de l'entité

3.3.1 Paramètres Génériques

Les paramètres génériques permettent de configurer facilement le comportement du système :

primaryblue!20 **Paramètre**	**Valeur**	**Signification**
CLK_HZ	100_000_000	Fréquence de l'horloge en Hz
DEBOUNCE_MS	10	Durée d'antirebond en millisecondes
SHOW_TIME_SEC	2	Durée d'affichage de la séquence
BLINK_PERIOD_MS	250	Période de clignotement (250ms)
BLINK_TOGGLES	6	Nombre de basculements (3 clignotements)

Table 3.1: Paramètres génériques du système

3.4 Constantes Calculées

```
constant DEB_CYCLES : integer := (CLK_HZ/1000) * DEBOUNCE_MS;
constant BLINK_CYCLES : integer := (CLK_HZ/1000) *
   BLINK_PERIOD_MS;
constant SHOW_CYCLES : integer := CLK_HZ * SHOW_TIME_SEC;
```

Listing 3.3: Calcul des constantes temporelles

Calcul des cycles d'horloge

Pour convertir un temps en nombre de cycles d'horloge :

$$\text{Nombre de cycles} = \text{Fréquence (Hz)} \times \text{Temps (s)} \tag{3.1}$$

Exemples :

- DEB_CYCLES $= (100 \times 10^6/1000) \times 10 = 1\,000\,000$ cycles

- SHOW_CYCLES $= 100 \times 10^6 \times 2 = 200\,000\,000$ cycles
- BLINK_CYCLES $= (100 \times 10^6/1000) \times 250 = 25\,000\,000$ cycles

3.5 Signaux Internes

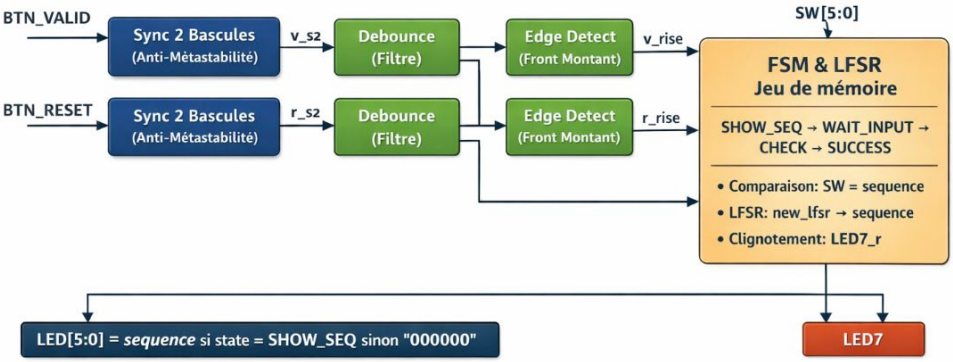


Figure 3.1: Flux de traitement des signaux : synchronisation, débounce et détection de front

3.5.1 Signaux de Synchronisation

```
signal v_s1, v_s2 : std_logic := '0';   -- VALID
signal r_s1, r_s2 : std_logic := '0';   -- RESET
```

Listing 3.4: Signaux pour l'anti-métastabilité

Problème de métastabilité

Les signaux asynchrones (boutons) peuvent changer pendant une période d'horloge, créant des états métastables indéterminés. La solution : utiliser deux bascules en série pour synchroniser le signal avec l'horloge.

3.5.2 Signaux d'Antirebond

```
signal v_db, r_db : std_logic := '0';
signal v_cnt : integer range 0 to DEB_CYCLES := 0;
signal r_cnt : integer range 0 to DEB_CYCLES := 0;
```

Listing 3.5: Signaux pour l'antirebond

3.5.3 Signaux de Détection de Front

```
signal v_db_d : std_logic := '0';
signal r_db_d : std_logic := '0';
signal v_rise : std_logic := '0';
signal r_rise : std_logic := '0';
```

Listing 3.6: Signaux pour la détection de front montant

3.5.4 Machine à États

```
type state_t is (SHOW_SEQ, WAIT_INPUT, CHECK, SUCCESS);
signal state : state_t := SHOW_SEQ;
```

Listing 3.7: Définition des états du jeu

primaryblue!20 **État**	**Description**
SHOW_SEQ	Affiche la séquence sur les LEDs pendant 2 secondes
WAIT_INPUT	Attend que l'utilisateur positionne les interrupteurs et valide
CHECK	Compare la réponse de l'utilisateur avec la séquence
SUCCESS	Fait clignoter LED7 trois fois puis génère une nouvelle séquence

Table 3.2: États de la machine à états finis

3.5.5 Signaux pour le LFSR et la Séquence

```
signal lfsr : std_logic_vector(5 downto 0) := "101001";
signal sequence : std_logic_vector(5 downto 0) := "101001";
```

Listing 3.8: Registres pour la génération aléatoire

3.6 Processus 1 : Anti-Métastabilité

```
-- 1) Synchronisation anti-metastabilite
process(CLK)
begin
    if rising_edge(CLK) then
        v_s1 <= BTN_VALID; v_s2 <= v_s1;
        r_s1 <= BTN_RESET; r_s2 <= r_s1;
    end if;
end process;
```

Listing 3.9: Synchronisation des signaux asynchrones

Double bascule (Double Flip-Flop)

Principe :

1. Le signal asynchrone est échantillonné par une première bascule
2. Le résultat passe par une deuxième bascule
3. Cela réduit drastiquement le risque de métastabilité

Chronogramme :

```
CLK     : ___|---|___|---|___|---|___|---|___
BTN     : __________|----------------|_______
v_s1    : _____________|-----------------|___
v_s2    : ____________________|---------------
```

3.7 Processus 2 : Antirebond (Debounce)

```
-- 2) Debounce (filtre)
process(CLK)
begin
    if rising_edge(CLK) then
        -- VALID
        if v_s2 = v_db then
            v_cnt <= 0;
        else
            if v_cnt = DEB_CYCLES then
                v_db <= v_s2;
                v_cnt <= 0;
            else
                v_cnt <= v_cnt + 1;
            end if;
        end if;

        -- RESET
        if r_s2 = r_db then
            r_cnt <= 0;
        else
            if r_cnt = DEB_CYCLES then
                r_db <= r_s2;
                r_cnt <= 0;
            else
                r_cnt <= r_cnt + 1;
            end if;
```

```
        end if;
    end if;
end process;
```

Listing 3.10: Filtrage des rebonds mécaniques

Problème des rebonds mécaniques

Lorsqu'on appuie sur un bouton mécanique, les contacts métalliques rebondissent plusieurs fois avant de se stabiliser, créant de multiples transitions électriques.
Solution : Attendre un temps défini (ici 10 ms) avant de considérer le changement comme stable.
Calcul : À 100 MHz, 10 ms représentent 1 000 000 cycles d'horloge.

3.8 Processus 3 : Détection de Front Montant

```
-- 3) Edge detect propre
process(CLK)
begin
    if rising_edge(CLK) then
        v_rise <= v_db and (not v_db_d);
        r_rise <= r_db and (not r_db_d);

        v_db_d <= v_db;
        r_db_d <= r_db;
    end if;
end process;
```

Listing 3.11: Détection des fronts montants

Détection de front montant

Condition : Un front montant est détecté lorsque :

$$\text{front_montant} = \text{signal_actuel} \wedge \neg\text{signal_précédent} \tag{3.2}$$

Détection de front montant

Condition : Un front montant est détecté lorsque :

$$\text{front_montant} = \text{signal_actuel} \wedge \neg\text{signal_précédent} \tag{3.2}$$

Table de vérité :

primaryblue!20 v_db_d	v_db	v_rise
0	0	0
0	1	1 (front montant détecté)
1	0	0
1	1	0

Cela permet de détecter **un appui unique** même si le bouton reste enfoncé.

Figure 3.2: Détection de front montant - Condition et table de vérité

Cela permet de détecter un **appui unique** même si le bouton reste enfoncé.

3.9 Processus 4 : LFSR et Machine à États

Ce processus gère la logique principale du jeu, incluant la génération de séquences aléatoires et la gestion des états.

3.9.1 LFSR (Linear Feedback Shift Register)

```
-- FSM + LFSR + clignotement
process(CLK)
    variable feedback : std_logic;
    variable new_lfsr : std_logic_vector(5 downto 0);
begin
    if rising_edge(CLK) then
        -- Sur RESET : nouvelle sequence + affichage 2s
        if r_rise = '1' then
            feedback := lfsr(5) xor lfsr(4);
            new_lfsr := lfsr(4 downto 0) & feedback;
            lfsr <= new_lfsr;
            sequence <= new_lfsr;

            state <= SHOW_SEQ;
            show_cnt <= 0;
            led7_r <= '0';
            blink_cnt <= 0;
            toggles_cnt <= 0;
```

Listing 3.12: Génération de séquence pseudo-aléatoire

Principe du LFSR

Un LFSR (Linear Feedback Shift Register) est un registre à décalage avec une rétroaction linéaire utilisé pour générer des séquences pseudo-aléatoires.
Fonctionnement :

1. Calculer le feedback : `feedback = lfsr[5] XOR lfsr[4]`
2. Décaler le registre vers la gauche
3. Insérer le feedback à la position 0

Polynôme utilisé : $x^6 + x^5 + 1$
Exemple de séquence :

```
État initial : 101001
Itération 1  : 010011  (feedback = 1 XOR 0 = 1)
Itération 2  : 100111  (feedback = 0 XOR 1 = 1)
Itération 3  : 001111  (feedback = 1 XOR 0 = 1)
...
```

Propriété : Un LFSR de 6 bits peut générer jusqu'à $2^6 - 1 = 63$ états différents avant de se répéter.

3.9.2 Gestion des États

```
else
    case state is
        -- Afficher la sequence pendant 2 secondes
        when SHOW_SEQ =>
            led7_r <= '0';
            if show_cnt = SHOW_CYCLES then
                show_cnt <= 0;
                state <= WAIT_INPUT;
            else
                show_cnt <= show_cnt + 1;
            end if;
```

Listing 3.13: Machine à états - État SHOW_SEQ

État SHOW_SEQ :

- Affiche la séquence sur LED0-LED5
- Compte jusqu'à 200 000 000 cycles (2 secondes)
- Passe à l'état WAIT_INPUT

```
        -- Jouer (LED eteintes), attendre appui VALID
        when WAIT_INPUT =>
```

```
            led7_r <= '0';
            if v_rise = '1' then
                state <= CHECK;
            end if;
```

Listing 3.14: Machine à états - État WAIT_INPUT

État WAIT_INPUT :

- Les LEDs sont éteintes
- Le système attend l'appui sur BTN_VALID
- Dès la détection du front montant, passage à CHECK

```
        -- Verifier la reponse
        when CHECK =>
            if SW = sequence then
                state <= SUCCESS;
                led7_r <= '0';
                blink_cnt <= 0;
                toggles_cnt <= 0;
            else
                state <= WAIT_INPUT;
            end if;
```

Listing 3.15: Machine à états - État CHECK

État CHECK :

- Compare SW (interrupteurs) avec sequence (séquence mémorisée)
- Si égal : passage à SUCCESS
- Si différent : retour à WAIT_INPUT (nouvelle tentative)

```
        -- Clignotement LED7 puis nouvelle manche
        when SUCCESS =>
            if blink_cnt = BLINK_CYCLES then
                blink_cnt <= 0;
                led7_r <= not led7_r;

                if toggles_cnt = BLINK_TOGGLES - 1 then
                    -- fin de victoire => nouvelle sequence
                    toggles_cnt <= 0;
                    led7_r <= '0';

                    feedback := lfsr(5) xor lfsr(4);
                    new_lfsr := lfsr(4 downto 0) & feedback;
                    lfsr <= new_lfsr;
                    sequence <= new_lfsr;

```

```
                        state <= SHOW_SEQ;
                        show_cnt <= 0;
                    else
                        toggles_cnt <= toggles_cnt + 1;
                    end if;
                else
                    blink_cnt <= blink_cnt + 1;
                end if;
        end case;
    end if;
end if;
end process;
```

Listing 3.16: Machine à états - État SUCCESS

État SUCCESS :

- Fait clignoter LED7 avec une période de 250 ms
- Compte 6 basculements (3 clignotements complets)
- Génère une nouvelle séquence aléatoire
- Retourne à l'état SHOW_SEQ pour une nouvelle manche

3.10 Affectations de Sortie

```
-- LED0..LED5 : affichees seulement pendant SHOW_SEQ, sinon
   eteintes
LED <= sequence when state = SHOW_SEQ else (others => '0');
LED7 <= led7_r;
```

Listing 3.17: Gestion de l'affichage des LEDs

Affectation conditionnelle

Cette ligne utilise une affectation conditionnelle :

- **Si** state = SHOW_SEQ : LED affiche la séquence
- **Sinon** : LED = "000000" (toutes éteintes)

La notation `(others => '0')` est un moyen élégant en VHDL d'initialiser tous les bits d'un vecteur à '0'.

Chapter 4

Schematic - Vue Logique du Circuit

4.1 Présentation du Schematic

Le schematic (schéma logique) est la représentation graphique automatiquement générée par l'outil de synthèse Xilinx Vivado à partir du code VHDL. Il montre comment les éléments logiques (portes, multiplexeurs, bascules, registres) sont interconnectés pour réaliser les fonctionnalités décrites dans le code.

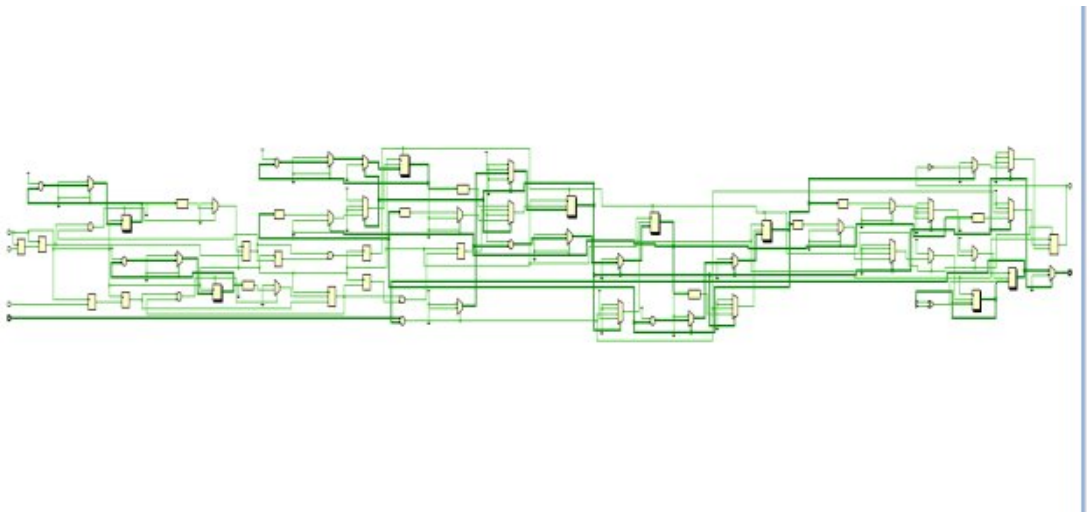


Figure 4.1: Schematic généré par Xilinx Vivado - Vue logique RTL du circuit

4.2 Rôle et Importance du Schematic

4.2.1 Vérification de la Synthèse

Le schematic permet de :

- **Vérifier la logique synthétisée** : S'assurer que le code VHDL a été correctement interprété
- **Identifier les optimisations** : Voir comment l'outil a simplifié ou réorganisé la logique

- **Détecter les problèmes** : Repérer les latches non intentionnels, les chemins combinatoires trop longs
- **Comprendre l'implémentation** : Visualiser comment les registres, compteurs et FSM sont réalisés

4.2.2 Éléments Visibles dans le Schematic

Dans notre schematic, on peut identifier :

primaryblue!20 **Élément**	**Correspondance dans le Code**
Registres (FD)	Signaux synchronisés sur rising_edge(CLK)
Multiplexeurs (MUX)	Instructions if-then-else, case-when
Portes logiques	Opérations AND, OR, XOR, NOT
Comparateurs	Conditions d'égalité (SW = sequence)
Compteurs	Signaux show_cnt, blink_cnt, v_cnt, r_cnt
FSM	Machine à états avec encodage des états

Table 4.1: Correspondance entre le code VHDL et les éléments du schematic

4.2.3 Analyse du Schematic

Le schematic montre la complexité réelle du circuit :

1. **Registres** : Tous les signaux avec `signal <= value` dans un process(CLK) deviennent des flip-flops
2. **Logique combinatoire** : Les opérations XOR du LFSR, les comparaisons, les conditions sont réalisées avec des portes logiques
3. **Multiplexage** : Les instructions if-else génèrent des multiplexeurs pour sélectionner les valeurs
4. **Optimisations** : L'outil peut fusionner ou simplifier certaines parties du code

Lecture du Schematic

Pour lire le schematic efficacement :

- Les rectangles représentent les registres (bascules D)
- Les trapèzes représentent les multiplexeurs
- Les petits symboles sont les portes logiques
- Les fils colorés montrent les bus de données
- Les signaux d'horloge et de reset sont généralement mis en évidence

Chapter 5

Concepts Avancés de Conception Numérique

5.1 Synchronisation et Métastabilité

5.1.1 Le Problème de la Métastabilité

Métastabilité : Un Danger Réel

Lorsqu'un signal asynchrone (comme un bouton) change de valeur exactement au moment où une bascule l'échantillonne, la bascule peut entrer dans un état métastable : ni '0' ni '1', mais un état intermédiaire instable.

Conséquences :

- Temps de propagation imprévisible
- Violation des contraintes de timing
- Comportement erratique du circuit
- Risque de propagation de l'erreur

5.1.2 Solution : Double Synchroniseur

Le double synchroniseur réduit exponentiellement la probabilité de métastabilité :

$$P_{\text{métastable}} \approx \frac{T_w \cdot f_{\text{data}} \cdot f_{\text{clk}}}{e^{t_r/\tau}} \tag{5.1}$$

Où :

- T_w : Fenêtre de métastabilité
- f_{data} : Fréquence du signal asynchrone
- f_{clk} : Fréquence de l'horloge
- t_r : Temps de résolution
- τ : Constante de temps de la bascule

5.2 Antirebond (Debouncing)

5.2.1 Phénomène de Rebond

Lorsqu'on appuie sur un bouton mécanique, les contacts ne se ferment pas instantanément. Ils rebondissent plusieurs fois sur une période de 5 à 20 ms.

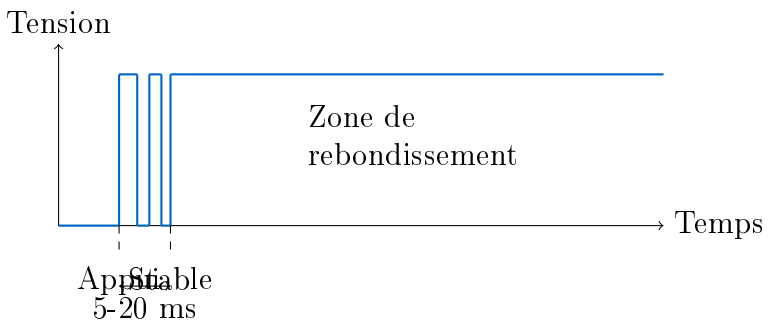


Figure 5.1: Phénomène de rebond mécanique

5.2.2 Algorithme d'Antirebond

Notre implémentation utilise un compteur pour filtrer les rebonds :

1. Si le signal change, on réinitialise le compteur
2. Si le signal reste stable, on incrémente le compteur
3. Après DEB_CYCLES (10 ms), on accepte la nouvelle valeur

5.3 Générateurs de Nombres Pseudo-Aléatoires

5.3.1 LFSR : Théorie

Un LFSR est déterministe mais produit une séquence qui semble aléatoire. La qualité de l'aléatoire dépend du polynôme choisi.

Polynômes primitifs pour différentes tailles :

primaryblue!20 **Bits**	**Polynôme**	**Période Max**
4	$x^4 + x^3 + 1$	15
6	$x^6 + x^5 + 1$	63
8	$x^8 + x^6 + x^5 + x^4 + 1$	255
16	$x^{16} + x^{15} + x^{13} + x^4 + 1$	65535

Table 5.1: Polynômes primitifs courants

5.3.2 Implémentation Matérielle

L'avantage du LFSR en matériel :

- **Très peu de ressources** : quelques portes XOR et des bascules
- **Rapidité** : génération en 1 cycle d'horloge
- **Déterminisme** : reproductible pour les tests

5.4 Machines à États Finis (FSM)

5.4.1 Diagramme d'États Complet

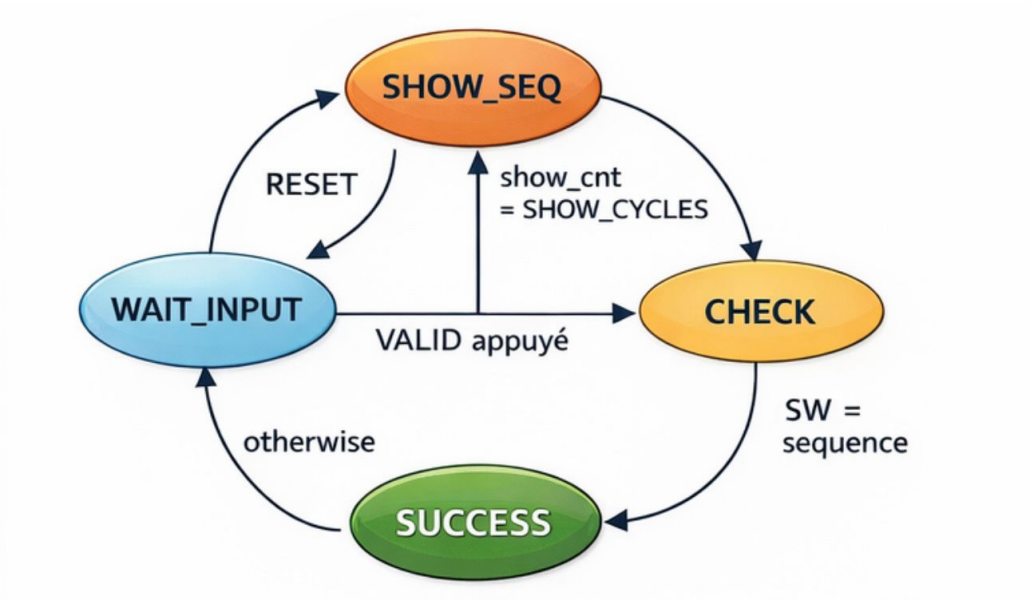


Figure 5.2: Diagramme de la machine à états finis du jeu Simon

5.4.2 Types de FSM

FSM de Moore vs FSM de Mealy :

primaryblue!20	**Moore**	**Mealy**
Sorties	Dépendent uniquement de l'état	Dépendent de l'état ET des entrées
Transitions	Plus stables	Plus rapides
Notre projet	(utilisé ici)	

Notre FSM est de type **Moore** car les sorties (LED) ne dépendent que de l'état actuel.

Chapter 6

Validation et Tests

6.1 Simulation Fonctionnelle

6.1.1 Banc de Test (Testbench)

Un testbench VHDL permet de vérifier le comportement du design avant de le programmer sur le FPGA.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity jeu_led_tb is
end jeu_led_tb;

architecture Behavioral of jeu_led_tb is
    -- Signaux de test
    signal clk : std_logic := '0';
    signal sw : std_logic_vector(5 downto 0) := "000000";
    signal btn_valid, btn_reset : std_logic := '0';
    signal led : std_logic_vector(5 downto 0);
    signal led7 : std_logic;

    -- Periode d'horloge
    constant clk_period : time := 10 ns;  -- 100 MHz
begin
    -- Instanciation du DUT (Device Under Test)
    uut: entity work.jeu_led
        generic map (
            CLK_HZ => 100_000_000,
            DEBOUNCE_MS => 1,  -- Reduit pour simulation
            SHOW_TIME_SEC => 1,
            BLINK_PERIOD_MS => 10
        )
        port map (
            CLK => clk,
            SW => sw,
```

```
29                BTN_VALID => btn_valid,
30                BTN_RESET => btn_reset,
31                LED => led,
32                LED7 => led7
33            );
34
35        -- Generation de l'horloge
36        clk_process: process
37        begin
38            clk <= '0';
39            wait for clk_period/2;
40            clk <= '1';
41            wait for clk_period/2;
42        end process;
43
44        -- Stimuli de test
45        stim_proc: process
46        begin
47            -- Reset initial
48            btn_reset <= '1';
49            wait for 100 ns;
50            btn_reset <= '0';
51
52            -- Attendre affichage sequence
53            wait for 1.1 sec;
54
55            -- Entrer une reponse correcte
56            sw <= led;  -- Copier la sequence
57            wait for 100 ns;
58            btn_valid <= '1';
59            wait for 100 ns;
60            btn_valid <= '0';
61
62            -- Observer le clignotement
63            wait for 500 ms;
64
65            wait;
66        end process;
67    end Behavioral;
```

Listing 6.1: Structure d'un testbench simplifié

6.2 Tests sur Carte FPGA

6.2.1 Procédure de Test

1. **Synthèse** : Convertir le VHDL en circuit logique
2. **Implémentation** : Placer et router sur le FPGA

3. **Génération du bitstream** : Créer le fichier de configuration
4. **Programmation** : Charger le bitstream sur le FPGA
5. **Vérification fonctionnelle** : Tester tous les scénarios

6.2.2 Scénarios de Test

primaryblue!20 **Test**	**Procédure**	**Résultat attendu**
1	Appuyer sur RESET	Séquence affichée 2s
2	Reproduire séquence correcte	LED7 clignote 3 fois
3	Reproduire séquence incorrecte	Retour à WAIT_INPUT
4	Appuyer plusieurs fois sur VALID	Un seul événement détecté
5	Nouvelle manche après succès	Nouvelle séquence différente

Table 6.1: Scénarios de validation

6.3 Analyse des Ressources

Après synthèse, le rapport Xilinx indique l'utilisation des ressources FPGA :

primaryblue!20 **Ressource**	**Utilisé**	**Disponible**	**% Util.**
LUT (Look-Up Tables)	∼150	63400	0.24%
Flip-Flops	∼80	126800	0.06%
I/O Pins	15	210	7.14%

Table 6.2: Utilisation des ressources FPGA (estimation)

Efficacité du Design

Notre design utilise moins de 1% des ressources du FPGA, ce qui est excellent et laisse beaucoup de place pour des améliorations futures.

Chapter 7

Améliorations et Extensions Possibles

7.1 Fonctionnalités Supplémentaires

7.1.1 Niveaux de Difficulté

- **Facile** : 4 bits, affichage 3 secondes
- **Moyen** : 6 bits, affichage 2 secondes (actuel)
- **Difficile** : 8 bits, affichage 1 seconde

7.1.2 Séquence Progressive

Au lieu d'une séquence fixe de 6 bits, implémenter une séquence qui s'allonge à chaque manche réussie (comme le Simon original).

```
signal sequence_length : integer range 1 to 10 := 1;
signal sequence_buffer : std_logic_vector(9 downto 0);

-- A chaque victoire
if state = SUCCESS then
    if sequence_length < 10 then
        sequence_length <= sequence_length + 1;
    end if;
end if;
```

Listing 7.1: Extension : séquence progressive

7.1.3 Compteur de Score

Ajouter un affichage 7 segments pour montrer le score (nombre de manches réussies).

7.1.4 Mode Multi-joueur

Implémenter deux ensembles de LEDs et interrupteurs pour permettre à deux joueurs de s'affronter.

7.2 Optimisations Techniques

7.2.1 LFSR Plus Complexe

Utiliser un LFSR de 32 bits pour une meilleure pseudo-aléatoire :

```
signal lfsr_32 : std_logic_vector(31 downto 0) := x"ACEBAD03";

-- Polynome : x^32 + x^22 + x^2 + x^1 + 1
feedback := lfsr_32(31) xor lfsr_32(21) xor lfsr_32(1) xor
   lfsr_32(0);
new_lfsr_32 := lfsr_32(30 downto 0) & feedback;
```

Listing 7.2: LFSR 32 bits

7.2.2 Communication UART

Ajouter une interface UART pour permettre la communication avec un PC et la sauvegarde des scores.

7.2.3 Sons et Effets

Utiliser un buzzer piézoélectrique pour jouer différentes fréquences correspondant à chaque LED.

7.3 Interface Graphique

7.3.1 Affichage VGA

Implémenter un contrôleur VGA pour afficher le jeu sur un écran :

- Animation de la séquence
- Score en grand format
- Menu de sélection de difficulté

Chapter 8

Conclusion

8.1 Synthèse du Projet

Ce projet de conception d'un jeu électronique Simon sur FPGA a permis d'explorer en profondeur les concepts fondamentaux de la conception de systèmes numériques. À travers cette implémentation, nous avons abordé :

- ✓ La **logique combinatoire** avec les opérations XOR du LFSR et la comparaison de séquences
- ✓ La **logique séquentielle** avec les registres, compteurs et machines à états
- ✓ La **gestion temporelle** avec la conversion temps/cycles et la génération de délais précis
- ✓ Le **traitement robuste des entrées** avec l'anti-métastabilité et l'antirebond
- ✓ La **génération pseudo-aléatoire** avec le LFSR
- ✓ Les **bonnes pratiques VHDL** : séparation entity/architecture, paramètres génériques, commentaires

8.2 Compétences Acquises

8.2.1 Compétences Techniques

- Maîtrise du VHDL
- Conception de FSM
- Gestion du timing
- Synchronisation de signaux
- Génération d'horloge
- Contraintes physiques (XDC)
- Simulation et validation
- Synthèse FPGA

8.2.2 Compétences Transversales

- Analyse de spécifications
- Conception modulaire
- Débogage méthodique
- Documentation technique
- Pensée algorithmique
- Résolution de problèmes

8.3 Applications Pratiques

Les concepts étudiés dans ce projet trouvent des applications dans de nombreux domaines :

primaryblue!20 **Domaine**	**Applications**
Télécommunications	Synchronisation de signaux, traitement de protocoles
Systèmes embarqués	Contrôleurs, interfaces homme-machine
Traitement du signal	Filtres numériques, génération de séquences
Cryptographie	Générateurs de nombres aléatoires, chiffrement
Automobile	ECU, systèmes de sécurité
Aéronautique	Systèmes critiques, redondance

Table 8.1: Applications des concepts du projet

8.4 Perspectives

Ce projet constitue une excellente base pour explorer davantage le monde de la conception numérique. Les prochaines étapes pourraient inclure :

1. Implémentation des améliorations proposées au chapitre 6
2. Étude de systèmes plus complexes (processeurs, DSP)
3. Exploration des outils de synthèse avancée
4. Optimisation pour la consommation énergétique
5. Conception de circuits ASIC
6. Apprentissage des langages HDL modernes (SystemVerilog, Chisel)

Figure 8.1: L'équipe du projet pendant la phase de développement et de tests sur FPGA

8.5 Réflexion Personnelle

La réalisation de ce projet démontre qu'un système apparemment simple cache en réalité une complexité fascinante. Chaque ligne de code VHDL se traduit en portes logiques physiques, et chaque décision de conception a un impact sur le matériel final.

Le FPGA offre une flexibilité exceptionnelle : le même circuit peut être reconfiguré pour implémenter des fonctions totalement différentes. Cette caractéristique en fait un outil incontournable pour le prototypage rapide et l'apprentissage de la conception numérique.

Citation

« La beauté de la conception FPGA réside dans la capacité de penser simultanément en termes de logique, de temps, et de matériel physique. C'est un art autant qu'une science. »

Appendix A

Code VHDL Complet

A.1 Fichier Principal : jeu_led.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity jeu_led is
generic (
    CLK_HZ : integer := 100_000_000;
    DEBOUNCE_MS : integer := 10;
    SHOW_TIME_SEC : integer := 2;
    BLINK_PERIOD_MS : integer := 250;
    BLINK_TOGGLES : integer := 6
);
port (
    CLK : in std_logic;
    SW : in std_logic_vector(5 downto 0);
    BTN_VALID : in std_logic;
    BTN_RESET : in std_logic;
    LED : out std_logic_vector(5 downto 0);
    LED7 : out std_logic
);
end entity;

architecture RTL of jeu_led is

    -- Constantes
    constant DEB_CYCLES : integer := (CLK_HZ/1000) * DEBOUNCE_MS
        ;
    constant BLINK_CYCLES : integer := (CLK_HZ/1000) *
        BLINK_PERIOD_MS;
    constant SHOW_CYCLES : integer := CLK_HZ * SHOW_TIME_SEC;

    -- Signaux de synchronisation
    signal v_s1, v_s2 : std_logic := '0';
```

```
    signal r_s1, r_s2 : std_logic := '0';

    -- Signaux d'antirebond
    signal v_db, r_db : std_logic := '0';
    signal v_cnt : integer range 0 to DEB_CYCLES := 0;
    signal r_cnt : integer range 0 to DEB_CYCLES := 0;

    -- Detection de front montant
    signal v_db_d : std_logic := '0';
    signal r_db_d : std_logic := '0';
    signal v_rise : std_logic := '0';
    signal r_rise : std_logic := '0';

    -- Machine a etats
    type state_t is (SHOW_SEQ, WAIT_INPUT, CHECK, SUCCESS);
    signal state : state_t := SHOW_SEQ;

    -- LFSR et sequence
    signal lfsr : std_logic_vector(5 downto 0) := "101001";
    signal sequence : std_logic_vector(5 downto 0) := "101001";

    -- Compteurs
    signal show_cnt : integer range 0 to SHOW_CYCLES := 0;
    signal blink_cnt : integer range 0 to BLINK_CYCLES := 0;
    signal toggles_cnt : integer range 0 to BLINK_TOGGLES := 0;
    signal led7_r : std_logic := '0';

begin

    -- LEDs affichees seulement pendant SHOW_SEQ
    LED <= sequence when state = SHOW_SEQ else (others => '0');
    LED7 <= led7_r;

    -- 1) Synchronisation anti-metastabilite
    process(CLK)
    begin
        if rising_edge(CLK) then
            v_s1 <= BTN_VALID; v_s2 <= v_s1;
            r_s1 <= BTN_RESET; r_s2 <= r_s1;
        end if;
    end process;

    -- 2) Debounce
    process(CLK)
    begin
        if rising_edge(CLK) then
            -- VALID
            if v_s2 = v_db then
                v_cnt <= 0;
```

```
            else
                if v_cnt = DEB_CYCLES then
                    v_db <= v_s2;
                    v_cnt <= 0;
                else
                    v_cnt <= v_cnt + 1;
                end if;
            end if;

            -- RESET
            if r_s2 = r_db then
                r_cnt <= 0;
            else
                if r_cnt = DEB_CYCLES then
                    r_db <= r_s2;
                    r_cnt <= 0;
                else
                    r_cnt <= r_cnt + 1;
                end if;
            end if;
        end if;
    end process;

    -- 3) Detection de front montant
    process(CLK)
    begin
        if rising_edge(CLK) then
            v_rise <= v_db and (not v_db_d);
            r_rise <= r_db and (not r_db_d);

            v_db_d <= v_db;
            r_db_d <= r_db;
        end if;
    end process;

    -- 4) FSM + LFSR + clignotement
    process(CLK)
        variable feedback : std_logic;
        variable new_lfsr : std_logic_vector(5 downto 0);
    begin
        if rising_edge(CLK) then
            -- Sur RESET : nouvelle sequence
            if r_rise = '1' then
                feedback := lfsr(5) xor lfsr(4);
                new_lfsr := lfsr(4 downto 0) & feedback;
                lfsr <= new_lfsr;
                sequence <= new_lfsr;

                state <= SHOW_SEQ;
```

```
                show_cnt <= 0;
                led7_r <= '0';
                blink_cnt <= 0;
                toggles_cnt <= 0;

            else
                case state is
                    when SHOW_SEQ =>
                        led7_r <= '0';
                        if show_cnt = SHOW_CYCLES then
                            show_cnt <= 0;
                            state <= WAIT_INPUT;
                        else
                            show_cnt <= show_cnt + 1;
                        end if;

                    when WAIT_INPUT =>
                        led7_r <= '0';
                        if v_rise = '1' then
                            state <= CHECK;
                        end if;

                    when CHECK =>
                        if SW = sequence then
                            state <= SUCCESS;
                            led7_r <= '0';
                            blink_cnt <= 0;
                            toggles_cnt <= 0;
                        else
                            state <= WAIT_INPUT;
                        end if;

                    when SUCCESS =>
                        if blink_cnt = BLINK_CYCLES then
                            blink_cnt <= 0;
                            led7_r <= not led7_r;

                            if toggles_cnt = BLINK_TOGGLES - 1 then
                                toggles_cnt <= 0;
                                led7_r <= '0';

                                feedback := lfsr(5) xor lfsr(4);
                                new_lfsr := lfsr(4 downto 0) & feedback;
                                lfsr <= new_lfsr;
                                sequence <= new_lfsr;

                                state <= SHOW_SEQ;
```

```
                                show_cnt <= 0;
                            else
                                toggles_cnt <= toggles_cnt + 1;
                            end if;
                        else
                            blink_cnt <= blink_cnt + 1;
                        end if;
                    end case;
                end if;
            end if;
        end process;

end architecture;
```

Listing A.1: Code VHDL complet du jeu Simon

Appendix B

Fichier de Contraintes

B.1 Fichier XDC Complet

```
# Configuration de l'horloge (100 MHz)
set_property IOSTANDARD LVCMOS33 [get_ports CLK]
set_property PACKAGE_PIN Y9 [get_ports CLK]

# Configuration des LEDs (sorties)
set_property IOSTANDARD LVCMOS33 [get_ports {LED[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {LED[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {LED[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {LED[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {LED[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {LED[5]}]

set_property PACKAGE_PIN T22 [get_ports {LED[0]}]
set_property PACKAGE_PIN T21 [get_ports {LED[1]}]
set_property PACKAGE_PIN U22 [get_ports {LED[2]}]
set_property PACKAGE_PIN U21 [get_ports {LED[3]}]
set_property PACKAGE_PIN V22 [get_ports {LED[4]}]
set_property PACKAGE_PIN W22 [get_ports {LED[5]}]

# Configuration de LED7
set_property IOSTANDARD LVCMOS33 [get_ports LED7]
set_property PACKAGE_PIN U14 [get_ports LED7]

# Configuration des interrupteurs (entrees)
set_property IOSTANDARD LVCMOS33 [get_ports {SW[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {SW[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {SW[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {SW[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {SW[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {SW[5]}]

set_property PACKAGE_PIN F22 [get_ports {SW[0]}]
set_property PACKAGE_PIN G22 [get_ports {SW[1]}]
```

```
set_property PACKAGE_PIN H22 [get_ports {SW[2]}]
set_property PACKAGE_PIN F21 [get_ports {SW[3]}]
set_property PACKAGE_PIN H19 [get_ports {SW[4]}]
set_property PACKAGE_PIN H18 [get_ports {SW[5]}]

# Configuration des boutons
set_property IOSTANDARD LVCMOS33 [get_ports BTN_VALID]
set_property PACKAGE_PIN N15 [get_ports BTN_VALID]

set_property IOSTANDARD LVCMOS33 [get_ports BTN_RESET]
set_property PACKAGE_PIN P16 [get_ports BTN_RESET]
```

Listing B.1: Fichier de contraintes XDC

Appendix C

Glossaire

ASIC Application-Specific Integrated Circuit - Circuit intégré conçu pour une application spécifique

Débounce Technique de filtrage des rebonds mécaniques d'un bouton

Entity Interface externe d'un composant VHDL (entrées/sorties)

FPGA Field-Programmable Gate Array - Circuit intégré reconfigurable

FSM Finite State Machine - Machine à états finis

HDL Hardware Description Language - Langage de description matériel

LFSR Linear Feedback Shift Register - Registre à décalage à rétroaction linéaire

LUT Look-Up Table - Table de correspondance programmable

Métastabilité État instable d'une bascule lors d'une violation des contraintes de timing

RTL Register Transfer Level - Niveau de transfert entre registres

Synthèse Processus de conversion du code HDL en circuit logique

VHDL VHSIC Hardware Description Language - Langage de description matériel

XDC Xilinx Design Constraints - Fichier de contraintes Xilinx

Appendix D

Bibliographie

Bibliography

[1] Peter J. Ashenden, *The Designer's Guide to VHDL*, Morgan Kaufmann, 3ème édition, 2008.

[2] Pong P. Chu, *FPGA Prototyping by VHDL Examples: Xilinx Spartan-3 Version*, Wiley-Interscience, 2008.

[3] M. Morris Mano, Michael D. Ciletti, *Digital Design: With an Introduction to the Verilog HDL*, Pearson, 5ème édition, 2012.

[4] Xilinx Inc., *Vivado Design Suite User Guide*, Documentation officielle Xilinx, 2024.

[5] Maxim Integrated, *Tutorial: Linear Feedback Shift Registers (LFSRs)*, Application Note, 2016.

[6] Peter Alfke, *Metastable Recovery in Virtex-II FPGAs*, Xilinx White Paper WP323, 2007.